How to use: paste this entire document into Claude (with the SportDrip repo attached) as the task prompt.

Prompt: Implement SportDrip's static engagement features

Your role

You are a senior full-stack engineer working in the **SportDrip** repository, a social sports-prediction app for iOS. You are implementing the remaining *static* engagement features: mechanics that do not need a content calendar once shipped. Implement them directly in the codebase, end to end (server, rules, indexes, client), not as a proposal.

Repository context

- **Monorepo:**
  - `artifacts/mobile/` is the Expo SDK ~58 React Native app (expo-router, reanimated, Firebase JS SDK, TypeScript strict). The `ios/` folder is checked in (prebuild).
  - `functions/src/` is the Firebase Cloud Functions v2 backend (TypeScript, zod, firebase-admin).
  - `firestore.rules` and `firestore.indexes.json` are at the repo root.
- **Path alias:** `@/` resolves to `artifacts/mobile/`.
- **Typecheck:**
  - App: `artifacts/mobile/node_modules/.bin/tsc --noEmit -p artifacts/mobile`
  - Functions: `cd functions && ../node_modules/.bin/tsc --noEmit -p .`
- **Tests:** `cd functions && npm test` (node:test against a fake Firestore in `functions/test/`). **Baseline: 131 pass / 2 fail.** The 2 failures are pre-existing (`getPlayerProfile` uses `.select()`, which the fake Firestore lacks). Do not introduce new failures.
- **Economy model:**
  - Free credits, no purchase of wager credits, no cash-out, cosmetics only.
  - League wallets use league tokens.
  - Every money mutation goes through `applyCredits` / transactions on the server.
  - The client never writes `users/{uid}` directly; it calls callables.

Already built (reuse, do not rebuild)

- **XP and levels:** `functions/src/lib/xp.ts` (`grantXp`, idempotent by key); `LevelBar` / `LevelUpHost` / `app/levels.tsx`.
- **Referrals:** `InviteFriendsCard` and `app/ref/[code].tsx`.

- **Achievements:** `functions/src/lib/achievements.ts` (`checkAchievements`, trophies `users/{uid}/trophies/ach_*`). The Locker shelf is in `app/(tabs)/avatar.tsx`.
- **Daily Pick:** `functions/src/callable/dailyPick.ts` (`chooseDailyPick`, `makeDailyPick`, `settleDailyPicksForGame`) and `components/DailyPickCard.tsx`.
- **Streak:** `functions/src/lib/streak.ts` (`advanceStreak` with freezes).
- **Weekly quests:** `functions/src/callable/quests.ts` and `components/QuestsCard.tsx`.
- **Crew feed:** `components/CrewFeed.tsx` and `app/feed.tsx`. **Game threads:** `components/GameThread.tsx` (league messages tagged `gameId`).
- **Reactions:**
  - league: `callable/social.ts` `reactToItem`, which notifies the owner
  - DMs: the `notifyOnDmReaction` trigger and the rules allowing a `reactors.<uid>` update
- **League chip and tail:**
  - "League is on this" chip: `useBoardLeagueSides` in `context/LeagueContext.tsx`, wired into `GameCard`.
  - Tail/fade: `placeLeagueBet` with `tailOf` / `fadeOf`. `GameQuickView` supports `follow`.
- **Push:**
  - `functions/src/triggers/push.ts` fans every notification doc out to Expo push, on create only.
  - `PushPrimer` component; the system push prompt is deferred until the first lock-in (`components/ToastHost.tsx`).
- **Server-side sweat alerts:** `functions/src/scheduled/sweat.ts`, opt-in via `users.sweatAlerts`.
- **Play limits:** `lib/playLimits.ts`, `setPlayLimits`, `setAlertPrefs`, `components/PlayLimitsCard.tsx`.
- **Prize boxes:** pity at `PITY_AFTER = 15` in `callable/prizes.ts`; published odds via `BOX_ODDS` in `constants/prizeTypes.ts`.
- **Onboarding** (`app/onboarding.tsx`): identity → appearance → teams → crew → pick → push primer → welcome.
- **Notification types** are a union in both `functions/src/lib/notifyModel.ts` (plus `notificationRoute`) and `artifacts/mobile/context/NotificationContext.tsx`. `app/activity.tsx` has an icon map that **must** include every type, or tsc fails.

Non-negotiable product guardrails

1. Never sell wager credits, and never let credits convert to anything of real-world value.
2. No in-play micro betting. Betting locks at game start.
3. Social surfaces show names, picks, accuracy and milestones. **Never stake size**, and never someone's loss unless they posted it.
4. Hidden picks (`hidden: true`, "hide until kickoff") must never reveal a side anywhere before kickoff.
5. Engagement pushes respect the per-user budget and quiet hours (item 7 below). Never send "come back and bet" after a loss, or to anyone on a break or at their daily limit.

6. No dark patterns: no fake scarcity, no countdown nags, neutral decline copy, and reduced motion respected.
7. Loans must not enable loss-chasing (the existing bank-loan cooldown stays).

Engineering conventions

- **Server:**
  - Validate every callable input with zod.
  - Writes are idempotent (deterministic doc ids or idempotency keys).
  - In transactions, do all reads before any write.
  - Side effects such as XP and achievements run *after* the money transaction commits.
- **Firestore:** add rules for every new collection; client writes are denied unless strictly needed. Add composite indexes to `firestore.indexes.json` and check for duplicates (a duplicate breaks deploy).
- **Client:**
  - Use the existing UI kit (`components/ui`: `Card`, `Pill`, `ActionChip`, `Button`, `SectionHeader`, `EmptyState`, `ScreenHeader`) and theme tokens from `useColors()`. No new hex literals.
  - Use `haptic` from `lib/motion`, plus accessibility labels on every pressable.
  - Wrap any AsyncStorage use in try/catch, as `lib/settings.ts` does.
- Keep diffs minimal and local, matching the surrounding style. Comments explain *why*, briefly.
- Add a unit test in `functions/test/` for new pure server logic (streak repair, cohort promotion, bracket scoring, budget classification).

The work

Implement the items below in the suggested order at the end. The original plan text follows. Treat each "Done when" and rule as an acceptance criterion.

Phase 1: small, code-only (≈1–2 weeks total)

1. League chat: rich bet cards with Tail (B3, A9)

- **Now:** league chat renders server `kind: "bet"` messages as plain rows. Tail exists only in `BetModal` / `GameQuickView` (`tailOf` / `fadeOf` on `placeLeagueBet`).
- **Build:**
  - Add a `BetChatCard` in `components/chat/`. It shows pick, odds and kickoff, plus a **Tail** button while the game is upcoming and the pick isn't hidden.
  - Tail opens the existing bet slip with `follow: { kind: "tail", betId, leagueId }`. `GameQuickView` already accepts this.

    - Render it in `app/messages.tsx` and `app/league/[id].tsx` for `kind === "bet"`.
- **Done when:** tailing from chat creates a `bet_tailed` activity and analytics event, and hidden picks show "Locked until kickoff" with no Tail.

2. Bet/result events raise chat banners (B3)

- **Now:** `context/ChatContext.tsx:335-360` only banners `kind === "text"`.
- **Build:**
    - Allow `kind === "bet"` from other members, at most one bet banner per league per hour (`notifiedMsgIds` plus a per-league timestamp map).
    - Keep system rows excluded.

3. Tail sheet from the "League is on this" chip (A3)

- **Now:** the chip renders but opens the generic detail sheet.
- **Build:**
    - Add an `onLeaguePress` prop to `GameCard` and a new `components/TailSheet.tsx`, fed by the same `gameSides` docs as `useBoardLeagueSides`.
    - List each member's name and side. Show "hidden" when `side === null` (hidden pick), never stake.
    - Each row has Tail / Fade into the bet slip with the league wallet preselected.
- **Note:** `useBoardLeagueSides` currently drops sides. Return them alongside names, and keep `GameCard` showing names only.

4. Streak completion (A8)

- **Repair path:**
    - In `lib/streak.ts`, if `diff === 2` and there are no freezes, set `repairDay = today`. The next qualifying action on the same day with `repairPending` restores `count + 1`, so two picks restore the streak.
    - Persist `repairPending: { day, needed: 2, done: n }`.
- **No-game days don't count:** `scheduleDailyPick` already knows when a day has no featured game. Write `dailyPick/{day}.empty = true`, and have `advanceStreak` treat a missed day with `empty === true` as neutral. It needs a lookup set passed in, or skip-day logic evaluated in the callable.
- **Milestone share card (7/30/100):** a `StreakMilestoneCard` view captured via `shareViewOrText`. Add `"streak"` to the `kind` union.
- **Weekly W/L grid:** build a 7-cell grid from `users/{uid}/dailyPicks` for the H2H week, share it to league chat as text (🟩🟥🟩⬜…), and offer it from `DailyPickCard` after a settled pick.

5. Celebration ladder (B8)

- Add `lib/celebrate.ts` with five rungs: `bet_win < h2h_week < streak_milestone < level_up < promotion`. Each rung maps to Burst count/power, LightBeams on/off, an avatar move, and

a share card on/off.
- Route `ResultPopup`, `LevelUpHost`, the streak toast and (later) promotion through it so intensity scales consistently. Reduced motion is already honored by the fx components.

6. First-7-days checklist (B6)

- Add a `FirstWeekChecklist` Home card shown while the account is younger than 7 days.
- **Items:**
  - avatar created (pre-checked, so progress starts endowed)
  - teams picked
  - first Daily Pick
  - a friend added
  - joined a league
  - first settled pick
- **State:** derive from the profile (`favoriteTeams`, `streak`, friends count, leagues). No new server state except `createdAt`, which already exists.
- **Activation metric:** log `activation_reached` when "in a league with ≥N friends + a settled pick" first becomes true. Tune N and X from cohorts, not now.

7. Notification budget and quiet hours (guardrail)

- **Server:** in `triggers/push.ts`, before sending:
  - Classify the type as **transactional** (results, challenges, loans, friend requests) or **engagement** (sweat, team/lock reminders, reactions, achievements).
  - Engagement pushes respect a per-user daily cap (default 4) and quiet hours (default 23:00–08:00 in the user's timezone; store `tz` on the user at sign-in).
  - Keep a counter in `users/{uid}/pushBudget/{day}`.
  - Never send "come back and bet" style copy after a loss. Already true; add a unit test.
- **Client:** add a "Quiet hours" toggle to the Locker "Alerts & limits" card, written through `setAlertPrefs`.

8. Prompt hygiene (B5)

- In `lib/promptGate.ts`, allow at most one system prompt per session (push, ATT or review).
- Move ATT in `context/AdsContext.tsx:83-95` to session ≥2 (store `sessionCount` in settings) and behind the gate.
- Never re-prompt after a decline.

9. Achievement badges in league tables (A11)

- Have `checkAchievements` also write `users/{uid}.achievementCount`.
- Standings already read member docs. Either mirror the count into `members.{uid}.achievementCount` when the trigger fires (one league update per active league), or read the user docs in the standings hook.

- Show a ⭐N chip in league standings rows and H2H cards.

10. Social quest: "React to 2 friends' picks" (A11)

- Replace the stand-in quest. In `callable/social.ts` `reactToItem` and the DM reaction trigger, increment `users/{uid}/questProgress/{week}.reactions` when a reaction is added (not removed) on someone else's item.
- `callable/quests.ts` reads that counter.

11. Measurement plumbing

- **Events:** extend `FUNNEL_EVENTS` in `lib/analytics.ts` with:
  - `slip_opened`, `slip_stake_set`, `slip_confirmed`, `slip_error`
  - `share_completed`, `reaction_sent`
  - `prompt_shown` / `prompt_accepted` / `prompt_declined` (with `prompt` param)
  - `daily_pick_made`, `quest_claimed`, `feed_item_tapped`, `tail_from_chat`
  - `activation_reached`
- **Experiments:** add `lib/experiments.ts` with deterministic bucketing, `hash(experimentId + unit) % 100`.
  - `unit` is the uid for individual features, or the **leagueId** for social features (A3, A4, A9, A10, A14) to avoid spillover.
  - A `globalHoldout` (e.g. 5%) disables new engagement surfaces.
  - The config doc `meta/experiments` holds `{ id, unit, splits, active }`. Treatment definitions live in code; only splits and on/off are config.
- **Server mirror:** `functions/src/lib/experiments.ts` uses the same hash so pushes and notifications respect arms.

12. Economy guardrails

- **Parlay share-of-balance cap:** in `callable/parlays.ts`, reject a ride stake above 25% of the current wallet. The leg cap `MAX_PARLAY_LEGS` already exists.
- **Bankrupt reset:** a `requestReset` callable for league members at 0 tokens with no active loans. It resets to 25% of starting tokens once per season, logs activity, and adds no debt.
- **Spin wheel staging:** in `components/SpinWheel.tsx`, redistribute easing so the final quarter-turn isn't most of the 5.6 s, and never land within one segment of the top prize when it didn't win (pick a landing angle from the winning segment's centre ± jitter only).

13. Live-game data freshness (A6 data)

- In `scheduled/syncGames.ts`, add a second schedule (every 30 s via two offset jobs, or a 1-minute job looping twice) that refreshes **only** games that are `live` and have open stakes (`sweat/{gameId}` exists or `bets.where(gameId).limit(1)`).
- Cost scales with live games, not users.
- Verify the actual provider cadence first (the "~2 min" figure came from a client comment).

14. Type scale and accessibility pass (B9)

- **Mechanical, screen by screen:**
  1. Raise everything below 11pt to ≥11 (start with odds tiles, wallet stats and ticket meta).
  2. Replace literal `fontSize` with `theme.type.*`.
  3. Replace hex literals with tokens.
  4. Set `maxFontSizeMultiplier` only on dense numeric tiles.
- **Accessibility labels:** add them to every `onPress` without one (148 of 326 today). Add an ESLint rule (`react-native-a11y/has-accessibility-props` or a custom rule) to stop regressions.
- **Live pulse:** move `GameCard`'s live pulse from `Animated` to reanimated with `useReducedMotion`.
- Track progress with a small script that counts literals per file.

15. Pick'em / Survivor audit

- The research flagged a discrepancy. Before scoping anything new, run each mode end to end (create → pick → settle → standings → season end) and list the gaps. `callable/picks.ts` and `app/league/[id].tsx:179-189` exist, so this is verification, not new work, unless gaps appear.

Phase 2: medium static features (≈3–5 weeks)

16. Avatar taunts and victory dances (A14)

- **Message:** a new league/DM message `kind: "taunt"` with `{ moveId, matchupId }`, posted by a callable (`sendTaunt`) that checks:
  - the sender **owns** the move (`users/{uid}/moves`)
  - at most 1 per matchup per day
  - the receiver hasn't muted taunts (`users/{uid}.muteTaunts`)
- **Rendering:** in lists, a still portrait (existing `AvatarPortraitStudio` stills). Tapping opens a modal `AvatarDisplay` playing `move={{ id }}`. No 3D mounted in chat lists. A pre-rendered video clip is a later optimisation.
- **Entry points:** "Send a dance" on win popups and H2H results, next to "Rub it in".

17. League co-op goals (A15)

- **Weekly goal:** `leagues/{id}/coop/{week}` with `target = clamp(5 × members, 20, 80)` correct picks, `progress`, and `rewardItemId`. The reward id comes from config `meta/coopRewards.current` (content is live-ops).
- **Progress:** incremented in `settleLeagueBetsForGame` on wins.
- **Completion:** grants the item to every member through the existing inventory write and posts a system chat row.

- **UI:** a contribution bar on the league page with per-member contribution.
- **Testing:** ship behind an experiment arm (item 11). The research says it's unproven.

18. Drip Wrapped (A16)

- **Generator:** `lib/wrapped.ts` builds `{ record, bestCall (highest odds won), worstBeat (biggest loss), rival (most H2H games), topTeam, streakBest, level, achievements }` from bets, league bets, rivalries and trophies.
- **Storage:** written to `users/{uid}/wrapped/{periodId}`, where the period is a league season id or `year-YYYY`.
- **Triggers:**
  - league end (`lib/leagueLifecycle.ts`)
  - a Dec 1 scheduled job for the yearly edition. The date is fixed, so the generation is static; promotional copy is live-ops.
- **Screen:** `app/wrapped/[id].tsx` in stories format (tap to advance), with each card shareable via `shareViewOrText`, plus an avatar dance on the final card.

19. Near-peer weekly ladder: promotion/relegation (A12, layer 2)

- **Tiers:** Bronze → Silver → Gold → Platinum → Diamond on `users/{uid}.ladder = { tier, cohortId, week }`.
- **Monday job (`scheduled/ladder.ts`):** builds cohorts of 10–15 by tier, seeding friends into the same cohort where possible. Score = weekly correct picks (Daily Pick + bets won).
- **Sunday close:** promote the top 3, relegate the bottom 3, write trophies, and fire the "promotion" celebration rung.
- **Screens:** a ladder screen with the cohort table framed against "players like you", and a tier badge on the avatar card and standings.
- **Gate:** turn on only when weekly actives ≥ ~200 (a config threshold). Below that, cohorts are too thin.

20. Season pass engine (A12, layer 1)

- **Static engine:**
  - `meta/seasons/{seasonId}` defines `{ startsAt, endsAt, tiers: [{ xp, reward }] }`.
  - Season XP is earned from quests and Daily Picks only, as a parallel counter to lifetime XP (`users/{uid}.seasonXp.{seasonId}`).
  - A `claimSeasonTier` callable grants the reward.
  - The pass screen shows the free track.
- **Rule baked into code:** the pass never replaces lifetime levels. Any premium track priced in credits can never award enough credits to buy the next pass; this is enforced in a validator test over the season config.
- Season content and cadence are in the live-ops doc.

21. Bracket league mode (A17)

- **New `mode: "bracket"`:**
  - `leagues/{id}/brackets/{uid}` holds picks per slot.
  - A `bracketTemplates/{templateId}` doc defines slots and feeds.
  - Scoring is 1-2-4-8-16-32 by round, settled from game results mapped to slots.
- **Static:** the engine, the bracket-fill UI (pinch/zoom grid) and scoring. **Live-ops:** creating templates for each real tournament.

Phase 3: native iOS (≈3–6 weeks, needs a dev build)

22. Live Activity per game with open bets (A6)

- **Target:** add a Widget Extension target with ActivityKit via a config plugin. `expo-apple-targets` is the usual path on Expo 58, as the repo already runs prebuild with `ios/` checked in.
- **Bridge:** a small native module (`modules/live-activity`) exposing `start(gameId, attrs)`, `update`, `end`, and the per-activity push token.
- **Start:** on lock-in (`ToastHost` `onLockedDone`). The bet confirm is the user action Apple requires.
  - Aggregate one activity per game: score, clock, your side, status pill, cover margin, H2H delta.
  - Limits are 8 h active, 4 KB payloads and no promotional content.
- **Updates:** Expo's push service does **not** send Live Activity updates. The server needs direct APNs (HTTP/2 + .p8 key in Secret Manager).
  - Store per-activity tokens at `users/{uid}/liveActivities/{gameId}`.
  - Send `apns-push-type: liveactivity` updates from the same sync path as sweat alerts.
  - End the activity on final.
- **Opt-in:** since iOS 17, push opt-in covers Live Activities. Also respect the in-app sweat toggle.

23. Home-screen widget (A7)

- **Placement:** in the same extension target, a WidgetKit small/medium widget showing this week's H2H score vs opponent, streak, and Daily Pick status.
- **Data:** an App Group shared `UserDefaults` written by the app (a native module or `react-native-shared-group-preferences`) whenever H2H, streak or Daily Pick state changes, then `WidgetCenter.reloadAllTimelines()`. Server-triggered refresh uses a silent push that wakes the app to rewrite the snapshot.
- **Promo:** shown after a win popup, not in onboarding. Randomise the promo (item 11) and read D30 by intent-to-treat.

Non-code checklist (do alongside)

- AdMob: block the "Social Casino Games" sensitive category.
- App Store: rate 17+/18+, and re-check the live Guidelines wording on sweepstakes and loot-box odds.
- Push copy review: no promotional content in Live Activities, and no post-loss re-engagement.

Suggested order

1, 2, 3 (social loop) → 7, 8 (protect the push channel before adding more pushes) → 11 (measure everything after) → 4, 5, 6, 9, 10 → 12, 13 → 14 (ongoing) → 16, 18, 20 → 17, 19, 21 → 22, 23.

How to work

1. **Read before writing.** For each item, open the files it names and confirm the current shape before editing. If the code has drifted from this prompt, follow the code and note the difference in your report.
2. **Track progress.** Work one item at a time with a task list. After each item, run both typechecks, and run `npm test` after any server change.
3. **Native items (22, 23):** - These need a dev build and an Apple Developer account. - Implement the config plugin, Swift extension, native module and server APNs sender. - Read secrets (APNs .p8 key, key id, team id) from Firebase Secret Manager via `defineSecret`, and never hard-code them. - If you cannot build or run iOS in your environment, still write the code, and list exactly what must be verified on device.
4. **Experiments:** ship anything user-facing that the plan marks experimental (co-op goals, widget promo, ladder) behind `lib/experiments.ts` (item 11). Build item 11 early.
5. **Ask only when blocked.** Ask only for decisions that are genuinely product calls, for example tier names or the ladder's minimum weekly actives. Otherwise choose the conservative option and record it.
6. **Do not:** touch the live-ops content items (shop rotation, season content, promotions, tournament templates); commit or push unless asked; or edit the pre-existing failing tests.

Final report (what to return when done)

- **Per item:** implemented / partial / skipped, the files changed, and any deviation from this spec with the reason.
- **Deploy checklist:** new functions, rules changes, new and removed indexes, secrets, and config docs to seed (e.g. `meta/experiments`, `meta/seasons/*` placeholders).

- **Device verification checklist** for anything you could not run.
- **Test results:** the typecheck output for both packages, and pass/fail counts versus the 131/2 baseline.